

JAVA BANK INTEREST PROGRAM

Object-Oriented Programming Practical Record – Program 7

1. Aim

To write a Java program using method overriding and runtime polymorphism to calculate interest for different banks.

2. Steps for Execution

1. Open VS Code, Eclipse or IntelliJ IDEA with JDK installed.
2. Create a Java file named **Program7.java**.
3. Create the parent class **Bank** with the `calculateInterest()` method.
4. Create **SBI** and **HDFC** classes that extend **Bank**.
5. Override `calculateInterest()` in both child classes.
6. Create a **Bank** reference variable in `main()`.
7. Assign **SBI** and **HDFC** objects to the **Bank** reference one after another.
8. Run the program and display the calculated interest.

3. Algorithm

1. Start the program.
2. Create the Bank parent class with calculateInterest().
3. Return 5% interest from the Bank implementation.
4. Create the SBI class and override calculateInterest() to return 6%.
5. Create the HDFC class and override calculateInterest() to return 7%.
6. In main(), set the amount to 10000.
7. Create a Bank reference variable b.
8. Assign an SBI object to b and calculate SBI interest.
9. Assign an HDFC object to b and calculate HDFC interest.
10. Display both interest amounts and stop the program.

4. Explanation of the Program

Bank class: The parent class contains calculateInterest(), which calculates 5% interest.

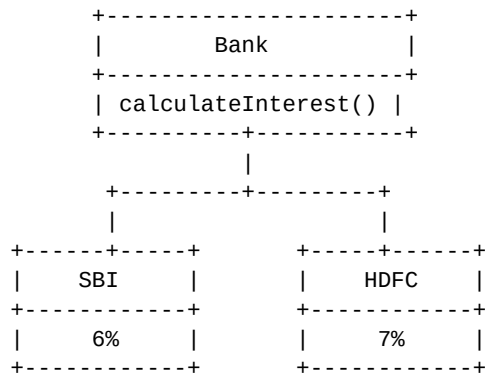
SBI class: Extends Bank and overrides calculateInterest() to calculate 6% interest.

HDFC class: Extends Bank and overrides calculateInterest() to calculate 7% interest.

Method Overriding: SBI and HDFC provide their own implementations of calculateInterest().

Runtime Polymorphism: The Bank reference b refers to different child objects, and the overridden method is selected at runtime.

5. UML Class Diagram



6. Java Source Code

```
class Bank {

    double calculateInterest(double amount) {
        return amount * 0.05;
    }
}

class SBI extends Bank {

    @Override
    double calculateInterest(double amount) {
        return amount * 0.06;
    }
}

class HDFC extends Bank {

    @Override
    double calculateInterest(double amount) {
        return amount * 0.07;
    }
}

public class Program7 {

    public static void main(String[] args) {

        double amount = 10000;

        Bank b;

        b = new SBI();
        System.out.println("SBI Interest = " +
            b.calculateInterest(amount));

        b = new HDFC();
        System.out.println("HDFC Interest = " +
            b.calculateInterest(amount));
    }
}
```

6. Java Source Code (continued)

```
class Bank {  
  
    double calculateInterest(double amount) {  
        return amount * 0.05;  
    }  
}  
  
class SBI extends Bank {  
  
    @Override  
    double calculateInterest(double amount) {  
        return amount * 0.06;  
    }  
}  
  
class HDFC extends Bank {  
  
    @Override  
    double calculateInterest(double amount) {  
        return amount * 0.07;  
    }  
}  
  
public class Program7 {  
  
    public static void main(String[] args) {  
  
        double amount = 10000;  
  
        Bank b;  
  
        b = new SBI();  
        System.out.println("SBI Interest = " +  
            b.calculateInterest(amount));  
  
        b = new HDFC();  
        System.out.println("HDFC Interest = " +  
            b.calculateInterest(amount));  
    }  
}
```

7. Output of the Program

```
SBI Interest = 600.0  
HDFC Interest = 700.0
```

8. Result

Thus, the Java program successfully demonstrates method overriding and runtime polymorphism by calculating different interest rates for SBI and HDFC.

Student Signature

Faculty Signature
